



Neural Network Stochastic Differential Equation Models

Ananda Biswas

M.Sc. Statistics and Computing, CIMS, BHU





Table of Contents

1 Discrete Deep Networks to Continuous Dynamics

► Discrete Deep Networks to Continuous Dynamics

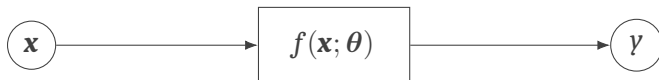
► Neural ODE

► Neural SDE



Deep Neural Networks

1 Discrete Deep Networks to Continuous Dynamics

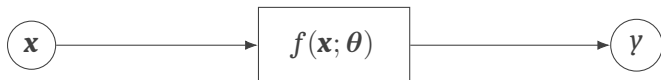


- $f(\cdot; \boldsymbol{\theta})$ is a deep neural network



Deep Neural Networks

1 Discrete Deep Networks to Continuous Dynamics

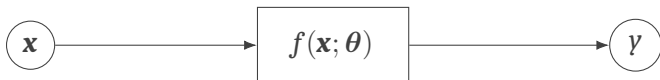


- $f(\cdot; \theta)$ is a deep neural network
- Learn θ from data



Deep Neural Networks

1 Discrete Deep Networks to Continuous Dynamics

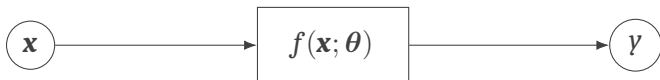


- $f(\cdot; \theta)$ is a deep neural network
- Learn θ from data
- Learning the residuals is often *easier* than learning the direct mapping



Deep Neural Networks

1 Discrete Deep Networks to Continuous Dynamics

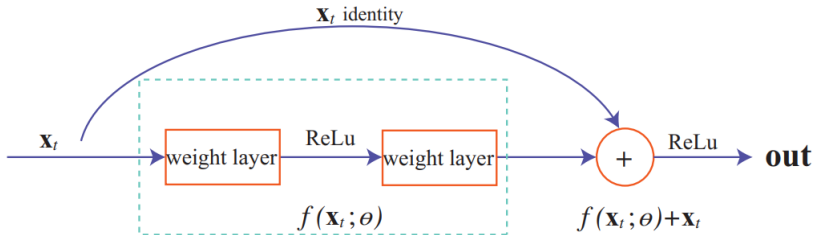


- $f(\cdot; \theta)$ is a deep neural network
- Learn θ from data
- Learning the residuals is often *easier* than learning the direct mapping
- Introduce Residual Networks



Residual Network

1 Discrete Deep Networks to Continuous Dynamics

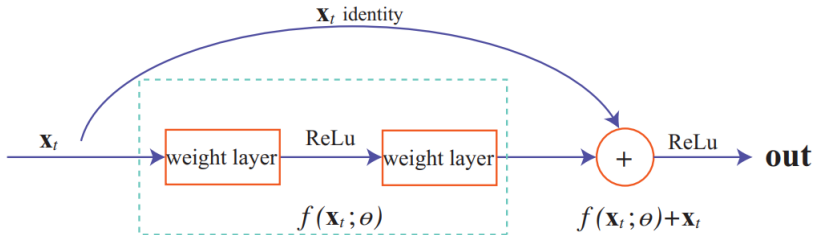


- The mapping of $f(\mathbf{x}_t; \theta)$ can be realised by feed-forward neural networks



Residual Network

1 Discrete Deep Networks to Continuous Dynamics

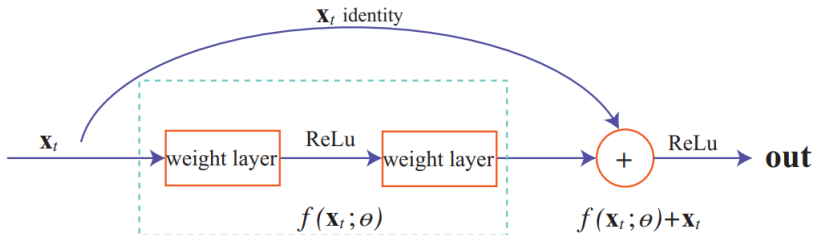


- The mapping of $f(\mathbf{x}_t; \theta)$ can be realised by feed-forward neural networks
- $\mathbf{x}_t$ is added to the output of the stacked layers



Residual Network

1 Discrete Deep Networks to Continuous Dynamics



- The mapping of $f(\mathbf{x}_t; \theta)$ can be realised by feed-forward neural networks
- $\mathbf{x}_t$ is added to the output of the stacked layers
- The hidden state $\mathbf{x}_t$ of the t -th layer of the network can be described by the equation

$$\mathbf{x}_{t+1} = \mathbf{x}_t + f(\mathbf{x}_t; \theta); \quad t = \{0, 1, 2, \dots, T\}.$$



ResNet to Ordinary Differential Equations

1 Discrete Deep Networks to Continuous Dynamics

Rewrite

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t f(\mathbf{x}(t); \boldsymbol{\theta}).$$



ResNet to Ordinary Differential Equations

1 Discrete Deep Networks to Continuous Dynamics

Rewrite

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t f(\mathbf{x}(t); \theta).$$

If we *add more layers* and take *small steps* i.e. $\Delta t \rightarrow 0$, then

$$\begin{aligned} \frac{\mathbf{x}(t + \Delta t) - \mathbf{x}(t)}{\Delta t} &= f(\mathbf{x}(t); \theta) \\ \Rightarrow \frac{d\mathbf{x}(t)}{dt} &= f(\mathbf{x}(t); \theta). \end{aligned}$$



ResNet to Ordinary Differential Equations

1 Discrete Deep Networks to Continuous Dynamics

Rewrite

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t f(\mathbf{x}(t); \theta).$$

If we *add more layers* and take small steps i.e. $\Delta t \rightarrow 0$, then

$$\begin{aligned} \frac{\mathbf{x}(t + \Delta t) - \mathbf{x}(t)}{\Delta t} &= f(\mathbf{x}(t); \theta) \\ \Rightarrow \frac{d\mathbf{x}(t)}{dt} &= f(\mathbf{x}(t); \theta). \end{aligned}$$

Thus we see that ResNet can indeed match the *Euler discretization* of ordinary differential equations.



Table of Contents

2 Neural ODE

► Discrete Deep Networks to Continuous Dynamics

► Neural ODE

► Neural SDE



Formalization

2 Neural ODE

A Neural Ordinary Differential Equation is given by

$$\frac{d\mathbf{x}(t)}{dt} = f(\mathbf{x}(t); \theta)$$

where $\mathbf{x}(t)$: hidden representation at time t ,
 $f(\cdot; \theta)$: a neural network,
 θ : parameters of the neural network f .



Forward Pass

2 Neural ODE

Given *initial state* $\mathbf{x}(0)$ and the differential equation, we compute

$$\mathbf{x}(T) = \mathbf{x}(0) + \int_0^T f(\mathbf{x}(t); \boldsymbol{\theta}) dt.$$

$f(\mathbf{x}(t); \boldsymbol{\theta})$ represents *how the state evolves at each instant*.



Backward Pass and Learning θ

2 Neural ODE

- Starting with an initial guess of θ , we calculate $x(T)$ using an ODE solver

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t f(\mathbf{x}(t); \theta).$$

This is a *Forward Pass* in practice.



Backward Pass and Learning θ

2 Neural ODE

- Starting with an initial guess of θ , we calculate $x(T)$ using an ODE solver

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t f(\mathbf{x}(t); \theta).$$

This is a *Forward Pass* in practice.

- Then compute the loss, calculate the derivatives w.r.t. the parameters. This is *Backward Pass*.



Backward Pass and Learning θ

2 Neural ODE

- Starting with an initial guess of θ , we calculate $x(T)$ using an ODE solver

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t f(\mathbf{x}(t); \theta).$$

This is a *Forward Pass* in practice.

- Then compute the loss, calculate the derivatives w.r.t. the parameters. This is *Backward Pass*.
- Update θ using the derivatives and repeat the process until convergence.



Table of Contents

3 Neural SDE

► Discrete Deep Networks to Continuous Dynamics

► Neural ODE

► Neural SDE



Formalization

3 Neural SDE

A Neural Stochastic Differential Equation is given by

$$d\mathbf{x}(t) = f(\mathbf{x}(t); \theta) dt + g(\mathbf{x}(t); \phi) dW_t$$

where $\mathbf{x}(t)$: hidden representation at time t ,

$f(\mathbf{x}(t); \theta)$: *drift* coefficient approximated by a neural network

dt : time increment

$g(\mathbf{x}(t); \phi)$: *diffusion* coefficient approximated by a neural network

dW_t : random noise increment



Forward Pass

3 Neural SDE

Based on the *Euler-Maruyama scheme*, a forward simulation of a SDE with Gaussian noise is approximated by

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + f(\mathbf{x}(t); \theta) \Delta t + g(\mathbf{x}(t); \phi) \sqrt{\Delta t} \epsilon_t.$$

- Two runs with same initialization produce different paths.



Forward Pass

3 Neural SDE

Based on the *Euler-Maruyama scheme*, a forward simulation of a SDE with Gaussian noise is approximated by

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + f(\mathbf{x}(t); \theta) \Delta t + g(\mathbf{x}(t); \phi) \sqrt{\Delta t} \epsilon_t.$$

- Two runs with same initialization produce different paths.
- We simulate many trajectories and take the average.



Backward Pass

3 Neural SDE

- Starting at an initial state $\mathbf{x}(0)$, with initial approximations of θ and ϕ , obtain $\mathbf{x}(T)$ by many small steps Δt .



Backward Pass

3 Neural SDE

- Starting at an initial state $\mathbf{x}(0)$, with initial approximations of θ and ϕ , obtain $\mathbf{x}(T)$ by many small steps Δt .
- Compute the loss and calculate the derivatives w.r.t. the parameters.



Backward Pass

3 Neural SDE

- Starting at an initial state $\mathbf{x}(0)$, with initial approximations of θ and ϕ , obtain $\mathbf{x}(T)$ by many small steps Δt .
- Compute the loss and calculate the derivatives w.r.t. the parameters.
- Update θ and ϕ using the derivatives and repeat until convergence.



Gaussian Noise

3 Neural SDE

- Depending on the nature of the random noise, there can be different classes of SDEs.



Gaussian Noise

3 Neural SDE

- Depending on the nature of the random noise, there can be different classes of SDEs.
- The most common random noise is *Gaussian Noise* under which

$$dW_t \sim \mathcal{N}(0, dt).$$



Gaussian Noise

3 Neural SDE

- Depending on the nature of the random noise, there can be different classes of SDEs.
- The most common random noise is *Gaussian Noise* under which

$$dW_t \sim \mathcal{N}(0, dt).$$

- In the forward simulation, we set

$$\epsilon_t \stackrel{iid}{\sim} \mathcal{N}(0, 1), \text{ consequently } \sqrt{\Delta t} \epsilon_t \stackrel{iid}{\sim} \mathcal{N}(0, \Delta t).$$



Non-Gaussian Noise

3 Neural SDE

- Gaussian noise is not always enough to capture big jumps.



Non-Gaussian Noise

3 Neural SDE

- Gaussian noise is not always enough to capture big jumps.
- Hence we introduce Non-Gaussian noise.



Non-Gaussian Noise

3 Neural SDE

- Gaussian noise is not always enough to capture big jumps.
- Hence we introduce Non-Gaussian noise.
- One such noise is α -Stable Lévy motion.



α -Stable Random Variables

3 Neural SDE

- A random variable X is said to be α -Stable if its n *i.i.d.* copies have the property

$$\sum_{i=1}^n X_i \stackrel{d}{=} n^{1/\alpha} X + b_n$$

and denoted as

$$X \sim \mathcal{S}_\alpha(\sigma, \beta, \gamma)$$

where σ : scale parameter, β : skewness, γ : location parameter.



α -Stable Random Variables

3 Neural SDE

- A random variable X is said to be α -Stable if its n *i.i.d.* copies have the property

$$\sum_{i=1}^n X_i \stackrel{d}{=} n^{1/\alpha} X + b_n$$

and denoted as

$$X \sim \mathcal{S}_\alpha(\sigma, \beta, \gamma)$$

where σ : scale parameter, β : skewness, γ : location parameter.

- $\sigma Z + \gamma \stackrel{d}{=} X$ where $Z \sim \mathcal{S}_\alpha(1, \beta, 0)$.



α -Stable Random Variables

3 Neural SDE

- A random variable X is said to be α -Stable if its n *i.i.d.* copies have the property

$$\sum_{i=1}^n X_i \stackrel{d}{=} n^{1/\alpha} X + b_n$$

and denoted as

$$X \sim \mathcal{S}_\alpha(\sigma, \beta, \gamma)$$

where σ : scale parameter, β : skewness, γ : location parameter.

- $\sigma Z + \gamma \stackrel{d}{=} X$ where $Z \sim \mathcal{S}_\alpha(1, \beta, 0)$.
- $X \sim \mathcal{S}_\alpha(1, 0, 0)$ is called a Standard α -Stable random variable.



α -Stable Random Variables

3 Neural SDE

- $\alpha \in (0, 2]$ controls how heavy tails are.



α -Stable Random Variables

3 Neural SDE

- $\alpha \in (0, 2]$ controls how heavy tails are.
- $\alpha = 2 \rightarrow$ Gaussian



α -Stable Random Variables

3 Neural SDE

- $\alpha \in (0, 2]$ controls how heavy tails are.
- $\alpha = 2 \rightarrow$ Gaussian
- $\alpha = 1 \rightarrow$ Cauchy



α -Stable Random Variables

3 Neural SDE

- $\alpha \in (0, 2]$ controls how heavy tails are.
- $\alpha = 2 \rightarrow$ Gaussian
- $\alpha = 1 \rightarrow$ Cauchy
- Smaller $\alpha \rightarrow$ heavier tails (more extreme values)



α -Stable Lévy Motions

3 Neural SDE

A symmetric α -Stable Lévy motion $\{L_t^\alpha\}_{t \geq 0}$ is a stochastic process with properties

- (i) $L_0^\alpha = 0$,
- (ii) L_t^α has independent increments,
- (iii) $L_t^\alpha - L_s^\alpha \sim \mathcal{S}_\alpha((t-s)^{1/\alpha}, 0, 0)$.



α -Stable Lévy Motions

3 Neural SDE

- For a very small step dt , $dL_{t+dt}^\alpha - dL_t^\alpha = dL_t^\alpha \sim S_\alpha((dt)^{1/\alpha}, 0, 0)$.



α -Stable Lévy Motions

3 Neural SDE

- For a very small step dt , $dL_{t+dt}^\alpha - dL_t^\alpha = dL_t^\alpha \sim S_\alpha((dt)^{1/\alpha}, 0, 0)$.
- We can write $(dt)^{1/\alpha} L_t \stackrel{d}{=} dL_t^\alpha$ where $L_t \sim S_\alpha(1, 0, 0)$.



α -Stable Lévy Motions

3 Neural SDE

- For a very small step dt , $dL_{t+dt}^\alpha - dL_t^\alpha = dL_t^\alpha \sim S_\alpha((dt)^{1/\alpha}, 0, 0)$.
- We can write $(dt)^{1/\alpha} L_t \stackrel{d}{=} dL_t^\alpha$ where $L_t \sim S_\alpha(1, 0, 0)$.
- In SDE, we set

$$L_t \stackrel{iid}{\sim} S_\alpha(1, 0, 0), \text{ consequently } (\Delta t)^{1/\alpha} L_t \stackrel{iid}{\sim} S_\alpha((\Delta t)^{1/\alpha}, 0, 0).$$



α -Stable Lévy Motions

3 Neural SDE

- For a very small step dt , $dL_{t+dt}^\alpha - dL_t^\alpha = dL_t^\alpha \sim S_\alpha((dt)^{1/\alpha}, 0, 0)$.
- We can write $(dt)^{1/\alpha} L_t \stackrel{d}{=} dL_t^\alpha$ where $L_t \sim S_\alpha(1, 0, 0)$.
- In SDE, we set

$$L_t \stackrel{iid}{\sim} S_\alpha(1, 0, 0), \text{ consequently } (\Delta t)^{1/\alpha} L_t \stackrel{iid}{\sim} S_\alpha((\Delta t)^{1/\alpha}, 0, 0).$$

- Euler-Maruyama approximation becomes

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + f(\mathbf{x}(t); \boldsymbol{\theta}) \Delta t + g(\mathbf{x}(t); \boldsymbol{\phi}) (\Delta t)^{1/\alpha} L_t.$$



Thank you !